

MongoDB Basic Practice Task

CRUD Operations, Query Operators & Real-World Library System

1. Objective

The objective of this practice task is to understand the basics of MongoDB, create databases and collections, perform CRUD operations, use query operators, and apply MongoDB concepts to a real-world use case.

2. Database & Collection Setup

```
Use studentDB  
db.createCollection("students") show collections
```

The **studentDB** database and **students** collection are created for storing student information.

```
test> use studentDB  
switched to db studentDB  
studentDB> db.createCollection("students")  
{ ok: 1 }  
studentDB> show collections  
students
```

3. Insert Operations

Insert one document:

```
db.students.insertOne({    name: "Ajay",    age: 21,    course: "MERN Stack",  
    status: "ongoing" })
```

```
studentDB> db.students.insertOne({  
    name: "Ajay",  
    age: 21,  
    course: "MERN Stack",  
    status: "ongoing"  
})  
  
acknowledged: true,  
insertedId: ObjectId('6abe6ffba86ebbacd9a8c274')
```

Insert multiple documents:

```
db.students.insertMany([ { name: "Arun", age: 22, course: "MERN Stack", status:
"completed" },
  { name: "Karthik", age: 20, course: "Java Full Stack", status: "ongoing" },
  { name: "Priya", age: 21, course: "MERN Stack", status: "ongoing" },
  { name: "Divya", age: 23, course: "Python Full Stack", status: "completed" }
])
```

```
studentDB> db.students.insertMany([
  {
    name: "Arun",
    age: 22,
    course: "MERN Stack",
    status: "completed"
  },
  {
    name: "Karthik",
    age: 20,
    course: "Java Full Stack",
    status: "ongoing"
  },
  {
    name: "Priya",
    age: 21,
    course: "MERN Stack",
    status: "ongoing"
  },
  {
    name: "Divya",
    age: 23,
    course: "Python Full Stack",
    status: "completed"
  }
])
```

4. Read Operations

Fetch all documents:

```
db.students.find() db.students.find().pretty()
```

```
students> db.students.find()
[
  {
    _id: ObjectId('6abf6fa0d3013135c68bbea5'),
    name: 'Ajay',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
students> db.students.find().pretty()
[
  {
    _id: ObjectId('6abf6fa0d3013135c68bbea5'),
    name: 'Ajay',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
```

Fetch students enrolled in MERN Stack:

```
db.students.find({ course: "MERN Stack" })
```

```
students> db.students.find({ course: "MERN Stack" })
[
  {
    _id: ObjectId('6abf6fa0d3013135c68bbea5'),
    name: 'Ajay',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6abf7273d3013135c68bbea7'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abf7273d3013135c68bbea9'),
    name: 'Priya',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
students> |
```

5. Update Operations

Update one student's status:

```
db.students.updateOne( { name: "Ajay" },  
  { $set: { status: "completed" } }  
)
```

```
students> db.students.updateOne( { name: "Ajay" },  
|   { $set: { status: "completed" } }  
| )  
|  
|  
{  
  acknowledged: true,  
  insertedId: null,  
  matchedCount: 1,  
  modifiedCount: 1,  
  upsertedCount: 0  
}  
students> |
```

Update multiple students:

```
db.students.updateMany( { course: "MERN Stack" },
  { $set: { status: "completed" } }
)
```

```
students> db.students.updateMany( { course: "MERN Stack" },
|   { $set: { status: "completed" } }
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 3,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
students> |
```

6. Delete Operations

Delete one document:

```
db.students.deleteOne({ name: "Karthik" })
```

```
students> db.students.deleteOne({ name: "Karthik" })
{ acknowledged: true, deletedCount: 1 }
students> |
```

Delete all documents for practice only:

```
db.students.deleteMany({})
```

```
students> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 4 }
students> |
```


7. Query Operators

Operator	Purpose	Example
\$gt	Find students older than 21.	db.students.find({ age: { \$gt: 21 } })
\$lt	Find students younger than 22.	db.students.find({ age: { \$lt: 22 } })
\$in	Find students in MERN Stack or Java Full Stack.	db.students.find({ course: { \$in: ["MERN Stack", "Java Full Stack"] } })
\$and	Find students older than 20 and completed.	db.students.find({ \$and: [{ age: { \$gt: 20 } }, { status: "completed" }] })
\$or	Find students in MERN Stack or Python Full Stack.	db.students.find({ \$or: [{ course: "MERN Stack" }, { course: "Python Full Stack" }] })
\$exists	Find documents where email exists.	db.students.find({ email: { \$exists: true } })
\$ne	Find students whose age is not equal to 21	db.students.find({ age: { \$ne: 21 } })

8. Real-World Use Case — Library System

A Library System can use MongoDB to store books, authors, categories, prices, and availability. The following commands demonstrate inserting, searching, and updating library data.

Create database and collection:

```
use libraryDB
db.createCollection("books")
```

```
students> use libraryDB
|
switched to db libraryDB
libraryDB> db.createCollection("books")
{ ok: 1 }
libraryDB> |
```

Insert books:

```
db.books.insertMany([ { title: "JavaScript Basics", author: "John Smith",  
category: "Programming", price: 450, available: true }, { title: "MongoDB Guide",  
author: "David Lee", category: "Database", price: 550, available: true }, {  
title: "React Fundamentals", author: "Robert James", category: "Programming",  
price: 600, available: false } ])
```

```
LibraryDB>  
| db.books.insertMany([ { title: "JavaScript Basics", author: "John Smith", category: "Programming", price: 450, available: true }, { title:  
"MongoDB Guide", author: "David Lee", category: "Database", price: 550, available: true }, { title: "React Fundamentals", author: "Robert Jam  
es", category: "Programming", price: 600, available: false } ])  
|  
{  
  acknowledged: true,  
  insertedIds: {  
    '0': ObjectId('6abf78bcd3013135c68bbeab'),  
    '1': ObjectId('6abf78bcd3013135c68bbeac'),  
    '2': ObjectId('6abf78bcd3013135c68bbead')  
  }  
}  
LibraryDB> |
```

Search programming books:

```
db.books.find({ category: "Programming" })
```

```
libraryDB> db.books.find({ category: "Programming" })
[
  {
    _id: ObjectId('6abe73e0a86ebbacd9a8c27d'),
    title: 'JavaScript Basics',
    author: 'John Smith',
    category: 'Programming',
    price: 450,
    available: true
  },
  {
    _id: ObjectId('6abe73e0a86ebbacd9a8c27f'),
    title: 'React Fundamentals',
    author: 'Robert James',
    category: 'Programming',
    price: 600,
    available: false
  },
  {
    _id: ObjectId('6abf78bcd3013135c68bbeab'),
    title: 'JavaScript Basics',
    author: 'John Smith',
    category: 'Programming',
    price: 450,
    available: true
  },
  {
    _id: ObjectId('6abf78bcd3013135c68bbead'),
    title: 'React Fundamentals',
    author: 'Robert James',
    category: 'Programming',
    price: 600,
    available: false
  }
]
libraryDB> |
```

Search books costing more than ■500:

```
db.books.find({ price: { $gt: 500 } })
```

```
libraryDB> db.books.find({ price: { $gt: 500 } })
[
  {
    _id: ObjectId('6abe73e0a86ebbacd9a8c27e'),
    title: 'MongoDB Guide',
    author: 'David Lee',
    category: 'Database',
    price: 550,
    available: false
  },
  {
    _id: ObjectId('6abe73e0a86ebbacd9a8c27f'),
    title: 'React Fundamentals',
    author: 'Robert James',
    category: 'Programming',
    price: 600,
    available: false
  },
  {
    _id: ObjectId('6abf78bcd3013135c68bbeac'),
    title: 'MongoDB Guide',
    author: 'David Lee',
    category: 'Database',
    price: 550,
    available: true
  },
  {
    _id: ObjectId('6abf78bcd3013135c68bbead'),
    title: 'React Fundamentals',
    author: 'Robert James',
    category: 'Programming',
    price: 600,
    available: false
  }
]
libraryDB> |
```

Update MongoDB Guide availability:

```
db.books.updateOne(      { title: "MongoDB Guide" },
  { $set: { available: false } }
)
```

```
libraryDB>
| db.books.updateOne(      { title: "MongoDB Guide" },
|   { $set: { available: false } }
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 0,
|   upsertedCount: 0
| }
libraryDB> |
```

9. Sample MongoDB Practice Screenshot

```
students> db.students.find({ age: { $gt: 21 } })
[
  {
    _id: ObjectId('6abe84dba86ebbacd9a8c284'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abe84dba86ebbacd9a8c287'),
    name: 'Divya',
    age: 23,
    course: 'Python Full Stack',
    status: 'completed'
  }
]
students> |
```

```

students> db.students.find({ age: { $lt: 22 }})
[
  {
    _id: ObjectId('6abe84dba86ebbacd9a8c285'),
    name: 'Karthik',
    age: 20,
    course: 'Java Full Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6abe84dba86ebbacd9a8c286'),
    name: 'Priya',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
students> |

```

```

students> db.students.find({ course: { $in: [ "MERN Stack", "Java Full Stack" ] } })
[
  {
    _id: ObjectId('6abe84dba86ebbacd9a8c284'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abe84dba86ebbacd9a8c285'),
    name: 'Karthik',
    age: 20,
    course: 'Java Full Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6abe84dba86ebbacd9a8c286'),
    name: 'Priya',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
students> |

```

```

students> db.students.find( { $or: [ { course:"MERN Stack"},{ course:"Python Full Stack"} ] } )
[
  {
    _id: ObjectId('6abe84dba86ebbacd9a8c284'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abe84dba86ebbacd9a8c286'),
    name: 'Priya',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6abe84dba86ebbacd9a8c287'),
    name: 'Divya',
    age: 23,
    course: 'Python Full Stack',
    status: 'completed'
  }
]
students> |

```

```

studentDB> db.students.find({
|   email: { $exists: true }
| })
[
  {
    _id: ObjectId('6abe714ca86ebbacd9a8c279'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'completed',
    email: 'arun@gmail.com'
  }
]

```

```

students> db.students.find(
|   $and: [
|       { age: { $gt: 20 } },
|       { status: "completed" }
|   ]
| })
[
  {
    _id: ObjectId('6abe84dba86ebbacd9a8c284'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abe84dba86ebbacd9a8c287'),
    name: 'Divya',
    age: 23,
    course: 'Python Full Stack',
    status: 'completed'
  }
]
students> |

```

```

students> db.students.find({ age: { $ne: 21 } })
[
  {
    _id: ObjectId('6abf70bed3013135c68bbea6'),
    name: 'Divya',
    age: 23,
    course: 'Python Full Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abf7273d3013135c68bbea7'),
    name: 'Arun',
    age: 22,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6abf7273d3013135c68bbeaa'),
    name: 'Divya',
    age: 23,
    course: 'Python Full Stack',
    status: 'completed'
  }
]
students> |

```


10. Conclusion

MongoDB is a NoSQL document database that stores data in flexible JSON-like documents. In this task, database and collection creation, insert, read, update, and delete operations were practiced. Query operators such as \$gt, \$lt, \$in, \$and, \$or, and \$exists were also used. The Library System demonstrates how MongoDB CRUD and search operations can be applied to a practical application.